



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Título del trabajo

Design and implementation of a modular IoT device

Alejandro Rodríguez Mederos

La Laguna, 12 de mayo de 2026

D.**Ivan Castilla Rodríguez**, profesor Titular de Universidad adscrito al Departamento de Nombre del Departamento de la Universidad de La Laguna, como tutor

D.**Santiago Torres Álvarez**, profesor Titular de Universidad adscrito al Departamento de Nombre del Departamento de la Universidad de La Laguna, como cotutor.

C E R T I F I C A (N)

Que la presente memoria titulada:

“Diseño e implementación de un dispositivo IoT modular”

ha sido realizada bajo su dirección por D./Dña **Nombre Apellido1 Apellido2**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 12 de mayo de 2026

Agradecimientos

XXX

XXX

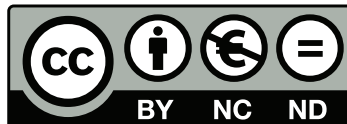
XXX

XXX

Licencia

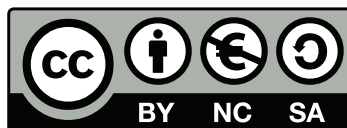
Elige una licencia y elimina el texto de opciones.

- Si NO quiere permitir que se compartan las adaptaciones de tu obra y NO quieres permitir usos comerciales de tu obra, indica:



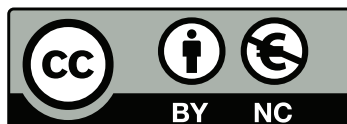
© Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-SinObraDerivada 4.0
Internacional.

- Si quiere permitir que se compartan las adaptaciones de tu obra mientras se comparta de la misma manera y NO quieres permitir usos comerciales de tu obra, indica:



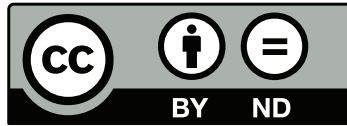
© Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-CompartirIgual 4.0
Internacional.

- Si quiere permitir que se compartan las adaptaciones de tu obra y NO quieres permitir usos comerciales de tu obra, indica:



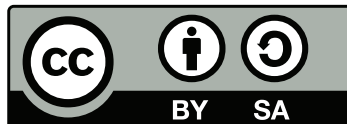
© Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial 4.0 Internacional.

- Si NO quiere permitir que se compartan las adaptaciones de tu obra y quieres permitir usos comerciales de tu obra, indica:



© Esta obra está bajo una licencia de Creative Commons Reconocimiento-SinObraDerivada 4.0 Internacional.

- Si quiere permitir que se compartan las adaptaciones de tu obra mientras se comparta de la misma manera y quieres permitir usos comerciales de tu obra (licencia de Cultura Libre) indica:



© Esta obra está bajo una licencia de Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional.

- Si quiere permitir que se compartan las adaptaciones de tu obra y quieres permitir usos comerciales de tu obra (licencia de Cultura Libre) indica:



© Esta obra está bajo una licencia de Creative Commons Reconocimiento 4.0 Internacional.

Resumen

El objetivo de este trabajo ha sido... bla, bla, bla, bla, bla, bla, bla, bla...

La competencia [E6], que figura en la guía docente, indica que en la memoria del trabajo se ha de incluir: antecedentes, problemática o estado del arte, objetivos, fases y desarrollo del proyecto, conclusiones, y líneas futuras.

Se ha incluido el apartado de 'Licencia' con todas las posibles licencias abiertas (Creative Commons). En el caso en que se decida hacer público el contenido de la memoria, habrá que elegir una de ellas (y borrar las demás). La decisión de hacer pública o no la memoria se indica en el momento de subir la memoria a la Sede Electrónica de la ULL, paso necesario en el proceso de presentación del TFG.

El documento de memoria debe tener un máximo de 50 páginas.

No se deben dejar páginas en blanco al comenzar un capítulo, ya que el documento no está pensado para ser impreso sino visionado con un lector de PDF.

También es recomendable usar márgenes pequeños, ya que, al firmar digitalmente por la Sede Electrónica, se coloca un marco alrededor del texto original.

El tipo de letra base ha de ser de 12 pt.

Palabras clave: Palabra reservada1, Palabra reservada2,...

Abstract

Here should be the abstract in a foreing language...

Keywords: Keyword1, Keyword2, Keyword3,...

Índice general

Capítulo 1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción de la solución	1
1.3. Tecnologías empleadas	2
1.4. Estructura del documento	2
Capítulo 2. Diseño del sistema	3
2.1. Visión general del sistema	3
2.2. Arquitectura modular	3
2.3. Arquitectura de procesamiento	4
2.4. Sistema de interacción	4
2.5. Sistema de sensorización	4
2.6. Almacenamiento y gestión de aplicaciones	5
2.7. Sistema de alimentación	5
Capítulo 3. Consumo Consumo Energético del Sistema	6
3.1. Escenarios de consumo	6
3.2. Cálculo de autonomía actualizado	7
3.3. Conclusión técnica actualizada	7
Capítulo 4. Diseño del lenguaje específico de dominio (DSL)	8
4.1. Introducción	8
4.2. Modelo general del lenguaje	8
4.3. Sintaxis general	9
4.4. Gestión de dispositivos	9
4.4.1. Dispositivos de salida	9
4.4.2. Dispositivos de entrada	9
4.5. Variables y asignación	10
4.5.1. Asignación directa	10
4.5.2. Asignación mediante operador flecha	10
4.6. Control de flujo	10
4.6.1. Bucles	10
4.6.2. Condicionales	11
4.7. Control temporal y salida de datos	11
4.7.1. Control temporal	11
4.7.2. Salida de datos	12
4.8. Comunicación I2C	12
4.9. Ejemplo completo de programa	12
4.10. Palabras clave y elementos del lenguaje	13
4.11. Consideraciones de diseño	13

4.12. Futuras mejoras	14
Capítulo 5. Diseño de la API	15
5.1. Introducción	15
5.2. Configuración Wi-Fi y modos de funcionamiento	15
5.3. Arquitectura de la Interfaz Web	17
5.4. Rutas principales del sistema	18
5.5. Dashboard principal	18
5.6. Gestión de archivos y edición de programas	18
5.7. Documentación integrada del DSL	19
5.8. Sistema de monitorización interna	19
5.9. Consideraciones de diseño	19
Capítulo 6. Desarrollo del software	21
Capítulo 7. Diseño Final del Dispositivo	24
7.1. Contexto del Diseño	24
7.2. Diseño Físico del Dispositivo	25
7.3. Diseño e Integración de Componentes Internos	26
7.3.1. Estructura interna y módulos principales	26
7.3.2. Gestión térmica	26
7.4. Comentarios sobre el Diseño	27
Capítulo 8. Summary and Conclusions	28
Capítulo 9. Presupuesto	29
9.1. Sección Uno	29
Apéndice A. Título del Apéndice 1	31
A.1. Algoritmo XXX	31
A.2. Archivo XXY	31
A.3. Algoritmo YYY	32
A.4. Algoritmo ZZZ	32
Apéndice B. Título del Apéndice 2	33
B.1. Otro apéndice: Sección 1	33
B.2. Otro apéndice: Sección 2	33

Índice de figuras

6.1. Representación conceptual del diseño del software.	21
7.1. Diseño interno del dispositivo. Distribución de módulos, sensores, tarjeta SD, ventilador y núcleo central ESP32-S3.	25

Índice de tablas

3.1. Resumen actualizado de consumos del sistema.	6
3.2. Autonomía estimada según modo de funcionamiento.	7
4.1. Clasificación de palabras clave y elementos del lenguaje.	13
5.1. Rutas principales disponibles en el sistema.	18
9.1. Presupuesto de Equipos y Licencias	29
9.2. Coste de Mano de Obra	29
9.3. Coste Total del Proyecto	30

Capítulo 1

Introducción

1.1. Contexto y motivación

En la actualidad, los sistemas basados en el Internet of Things (IoT) han experimentado un notable crecimiento, dando lugar a una amplia variedad de dispositivos diseñados para cubrir necesidades específicas en ámbitos como la automatización, la monitorización y el control de procesos. Sin embargo, muchas de estas soluciones están altamente especializadas, lo que implica la necesidad de utilizar diferentes dispositivos para cada aplicación concreta.

Esta situación genera una dependencia de soluciones cerradas y poco flexibles, dificultando la reutilización de hardware y aumentando tanto el coste como la complejidad de implementación de nuevos sistemas.

Sumado a esto se encuentra el problema de tener que investigar sobre los diferentes módulos que existen, cual viene mejor para el proyecto que quieres montar o inclusive si simplemente quieres adentrarte en el mundo del IoT. Es aquí donde surge la necesidad de crear un sistema que permita al usuario crear de manera rápida y sencilla programas y diseños rápidos con los que poder trabajar de manera casi inmediata. Es aquí donde surge el presente Trabajo de Fin de Grado, buscando crear un dispositivo modular, flexible y adaptable a distintos entornos de uso. Evitando que el usuario tenga que conectar cables con la posibilidad de tener errores, documentarse sobre un código complejo, o simplemente reducir su tiempo de desarrollo y poner en práctica su idea.

De forma más concreta, el proyecto persigue los siguientes objetivos:

- Diseñar un dispositivo base reutilizable para múltiples aplicaciones.
- Permitir la ampliación de funcionalidades mediante módulos adicionales.
- Facilitar la creación de herramientas de monitorización, gestión y automatización.
- Reducir el tiempo y los recursos necesarios para desarrollar soluciones específicas.

1.2. Descripción de la solución

Para alcanzar estos objetivos, se propone una arquitectura modular que permite la incorporación de nuevos componentes funcionales de forma sencilla, ampliando las capacidades del sistema de manera prácticamente inmediata y reduciendo su complejidad lo máximo posible.

Asimismo, el sistema integra un lenguaje de instrucciones propio que posibilita la creación de programas de forma rápida e intuitiva, sin necesidad de desarrollar firmware específico para cada aplicación. Complementariamente, se dispone de una interfaz web que permite tanto la configuración del sistema como la visualización de registros (*logs*) y la programación de tareas.

1.3. Tecnologías empleadas

La implementación del sistema se basa en el uso de un microcontrolador **ESP32**, el cual proporciona capacidades de procesamiento, conectividad Wi-Fi y soporte para múltiples periféricos. Este se complementa con distintos módulos hardware que permiten ampliar las funcionalidades del dispositivo, los cuales serán descritos en detalle en capítulos posteriores.

1.4. Estructura del documento

El presente documento se organiza de la siguiente manera:

- En el Capítulo 2 se describe el diseño general del sistema y su arquitectura.
- En el Capítulo 3 se detallan los componentes hardware utilizados.
- En el Capítulo 4 se analiza el consumo energético y la autonomía del dispositivo.
- En el Capítulo 5 se aborda la implementación software del sistema.
- En el Capítulo 6 se aborda la implementación del DSL (Domain Specific Language).
- Finalmente, en el Capítulo 6 se presentan las conclusiones y posibles líneas de trabajo futuro.

Capítulo 2

Diseño del sistema

2.1. Visión general del sistema

El sistema desarrollado consiste en una plataforma IoT modular diseñada para facilitar la creación de soluciones de monitorización, automatización y gestión de forma flexible y reutilizable.

El dispositivo integra múltiples elementos hardware que permiten tanto la interacción directa con el usuario como la adquisición de datos del entorno. Entre estos componentes se incluyen una pantalla para visualización de información, indicadores luminosos (LEDs), un zumbador, una unidad de medida inercial (IMU) y un sistema de expansión de pines (I2C) que facilita la incorporación de nuevos sensores.

Todo el sistema se encuentra controlado por un microcontrolador ESP32 Dual-Core, que actúa como unidad central de procesamiento. Además, se incorpora un sistema de almacenamiento basado en tarjeta microSD, permitiendo almacenar y gestionar diferentes aplicaciones, así como intercambiar configuraciones de forma rápida.

Desde el punto de vista energético, el dispositivo se alimentará mediante un conjunto de baterías de litio, cuya gestión se realiza mediante circuitos dedicados que permiten tanto el balanceo como la correcta carga de las mismas, garantizando un funcionamiento seguro y eficiente.

2.2. Arquitectura modular

Uno de los aspectos clave del sistema es su enfoque modular. Este se basa en la utilización de un bus de expansión I2C que permite la conexión de dispositivos externos de forma sencilla.

Gracias a este diseño, es posible incorporar nuevos sensores y módulos sin necesidad de modificar la arquitectura base del sistema, lo que proporciona una alta escalabilidad y flexibilidad. Esta característica responde directamente a uno de los objetivos principales del proyecto: permitir la adaptación del dispositivo a distintos escenarios sin rediseñar el hardware completo.

Adicionalmente, esta modularidad se encuentra soportada a nivel software mediante el lenguaje de instrucciones integrado en el sistema, el cual permite gestionar dinámicamente los dispositivos conectados.

2.3. Arquitectura de procesamiento

El sistema hace uso de la arquitectura de doble núcleo del ESP32 para separar responsabilidades y mejorar la eficiencia del sistema.

Uno de los núcleos se encarga de la ejecución de la lógica principal del sistema, incluyendo la gestión del lenguaje de instrucciones, el control del flujo de ejecución y la coordinación general del dispositivo. El segundo núcleo se dedica a la gestión de los módulos externos, permitiendo una ejecución concurrente que mejora el rendimiento y la capacidad de respuesta del sistema.

Esta división permite desacoplar las tareas críticas del sistema de las operaciones relacionadas con la entrada/salida, aumentando la robustez y escalabilidad del dispositivo.

2.4. Sistema de interacción

El dispositivo incorpora diversos elementos que permiten la interacción con el usuario y la visualización del estado del sistema.

Por un lado, la pantalla integrada permite mostrar información relevante como aplicaciones disponibles o datos capturados por los sensores (*logs*) haciendo uso del DSL. Esto facilita la supervisión directa del dispositivo sin necesidad de herramientas externas.

Todo esto viene soportado también por la aplicación web, donde dispondremos de un dashboard para gestionar los archivos cargados en el sistema.

Por otro lado, se dispone de tres LEDs principales con distintas funciones:

- **LED de conectividad Wi-Fi:** indica si el dispositivo se encuentra conectado a una red Wi-Fi.
- **LED de modo de operación:** muestra el estado del sistema, diferenciando, por ejemplo, entre modo normal y modo de desarrollo, de los cuales se profundizará más tarde.
- **LED configurable:** puede ser controlado por el usuario o por las aplicaciones, permitiendo su uso como indicador personalizado o señal de control.

2.5. Sistema de sensorización

El dispositivo integra una unidad de medida inercial (IMU) compuesta por acelerómetro, giroscopio y magnetómetro. Este conjunto de sensores permite registrar movimientos, vibraciones y variaciones en la orientación, así como cambios en el campo magnético del entorno.

Gracias a esta información, el sistema es capaz de determinar su orientación en el espacio, lo que habilita funcionalidades avanzadas como la monitorización de movimiento, detección de eventos o calibración automática de otros sensores.

2.6. Almacenamiento y gestión de aplicaciones

El uso de una tarjeta microSD como sistema de almacenamiento permite dotar al dispositivo de una alta flexibilidad en la gestión de aplicaciones.

Cada tarjeta puede contener diferentes configuraciones o programas, lo que facilita el cambio rápido entre distintos usos del dispositivo sin necesidad de reprogramar el firmware completamente. Este enfoque refuerza la idea de sistema reutilizable y adaptable a múltiples escenarios.

2.7. Sistema de alimentación

El sistema de alimentación se basa en el uso de baterías de litio, gestionadas mediante circuitos específicos encargados de regular la carga, descarga y balanceo entre celdas.

Este diseño garantiza un funcionamiento estable del dispositivo, al tiempo que protege las baterías frente a condiciones de operación no seguras, contribuyendo a la durabilidad del sistema.

Capítulo 3

Consumo Consumo Energético del Sistema

A partir de los componentes actualmente integrados en el sistema, se realiza una estimación del consumo energético considerando valores típicos en diferentes modos de operación. Se agrupan los dispositivos por tipo funcional en lugar de modelo específico.

Componente	Mínimo (mA)	Máximo (mA)	Notas
Microcontrolador (ESP32)	0.007	340	Wi-Fi activo en transmisión máxima.
Pantalla (controlador)	0.1	6	Sin retroiluminación.
Retroiluminación pantalla	0	40	Dependiente de brillo (PWM).
Sensor inercial (IMU)	0.1	15	Según tasa de muestreo.
Almacenamiento (microSD)	0.05	100	Picos en escritura.
Gestión de alimentación	0.01	0.02	Consumo quiescente.
Expansor de pines	0.1	1	Dependiente de uso.
LEDs indicadores (x3)	0	30	Dependiente del estado.
Zumbador	0	30	Solo en activación.
Total Estimado	=0.4	=560–590	

Tabla 3.1: Resumen actualizado de consumos del sistema.

3.1. Escenarios de consumo

Para obtener una estimación más realista del comportamiento del sistema, se consideran distintos escenarios de funcionamiento:

- **Modo bajo consumo:** La pantalla se encuentra apagada para reducir el consumo.
 - Consumo estimado: **1–5 mA**
- **Modo normal:** servidor activo, sensores funcionando junto con la pantalla.
 - Consumo estimado: **180–260 mA**
- **Modo alto rendimiento:** transmisión de datos, escritura en SD, sensores externos activos.
 - Consumo estimado: **400–590 mA**

3.2. Cálculo de autonomía actualizado

Se mantiene un factor de pérdida del 20 % para tener en cuenta ineficiencias del sistema:

$$\text{Usable (mAh)} = \text{Capacidad nominal} \times 0,8$$

$$\text{Autonomía (h)} = \frac{\text{Usable (mAh)}}{\text{Consumo medio (mA)}}$$

Capacidad (mAh)	Modo bajo	Modo normal	Modo alto
2000	320–1600 h	6–8 h	3–4 h
5000	800–4000 h	15–22 h	8–12 h
10000	1600–8000 h	30–44 h	16–24 h

Tabla 3.2: Autonomía estimada según modo de funcionamiento.

3.3. Conclusión técnica actualizada

El consumo del sistema está fuertemente condicionado por el uso del servidor, la retransmisión de datos, la retroiluminación de la pantalla, las operaciones de escritura en la memoria externa junto con el uso de sensores externos.

En condiciones normales de operación, el dispositivo presenta un consumo medio comprendido entre **180 y 260 mA**, lo que permite autonomías cercanas a las 24 horas con baterías de alta capacidad.

La implementación de modos de bajo consumo, como la desactivación selectiva de periféricos o el uso de *deep sleep*, permite aumentar significativamente la autonomía del sistema, superando varios días de funcionamiento continuo en escenarios de baja actividad.

Todos estos datos representan un cálculo estimado del uso del dispositivo dado que al no contar con los materiales necesarios para llevar a cabo la implementación del hardware correspondiente a la batería no se han podido realizar pruebas específicas, aportando así datos reales.

Capítulo 4

Diseño del lenguaje específico de dominio (DSL)

4.1. Introducción

El sistema desarrollado incorpora un lenguaje específico de dominio (*Domain Specific Language* o DSL) diseñado para la configuración, el control y la monitorización del dispositivo IoT.

Este lenguaje permite definir el comportamiento del sistema mediante instrucciones interpretadas en tiempo de ejecución, evitando la necesidad de recompilar el firmware cada vez que se desea modificar la lógica de funcionamiento. Gracias a ello, se obtiene una mayor flexibilidad durante el desarrollo y una reducción significativa en los tiempos de despliegue de nuevas funcionalidades.

El objetivo principal del diseño ha sido proporcionar una interfaz textual ligera, intuitiva y cercana al hardware, permitiendo interactuar con los distintos módulos del sistema de manera sencilla y eficiente.

4.2. Modelo general del lenguaje

El lenguaje se basa en un modelo de ejecución secuencial interpretado. Cada línea del programa se procesa en orden, salvo en estructuras de control como bucles o condicionales, que modifican el flujo normal de ejecución.

El procesamiento del código se divide en dos fases principales:

- **Análisis léxico (Lexer):** encargado de transformar el código fuente en una secuencia de tokens, identificando palabras clave, identificadores, operadores y valores numéricos.
- **Ejecución (Intérprete):** responsable de procesar los tokens generados, mantener el estado interno del sistema y gestionar dispositivos, variables y periféricos registrados.

Esta separación entre análisis y ejecución facilita la mantenibilidad del sistema y permite ampliar el lenguaje de forma modular sin modificar la arquitectura principal.

4.3. Sintaxis general

El lenguaje ha sido diseñado siguiendo criterios de simplicidad y legibilidad. La sintaxis está basada en instrucciones directas y estructuras de tipo clave-valor, reduciendo la complejidad de aprendizaje y facilitando su uso en sistemas embebidos.

Además, las palabras clave del lenguaje no distinguen entre mayúsculas y minúsculas, mejorando la usabilidad y reduciendo posibles errores de escritura.

4.4. Gestión de dispositivos

4.4.1. Dispositivos de salida

Actualmente, el sistema soporta dispositivos de salida de tipo led y buzzer. Cada dispositivo debe declararse previamente indicando el pin físico al que se encuentra asociado.

```
DEVICE=led NAME=myled pin=17
DEVICE=buzzer NAME=mybuz pin=33
```

Una vez registrado el dispositivo, su estado puede modificarse mediante la instrucción WRITE:

```
WRITE=myled on
WRITE=myled off
```

4.4.2. Dispositivos de entrada

Los dispositivos de entrada están limitados actualmente al tipo button, permitiendo la lectura de señales digitales simples. Su declaración sigue la misma estructura utilizada para los dispositivos de salida:

```
DEVICE=button NAME=mybutton pin=33
```

El estado de estos dispositivos puede consultarse continuamente dentro de estructuras iterativas. Debido al funcionamiento del intérprete, la monitorización constante de entradas requiere ejecutarse dentro de un bucle infinito.

```
LOOP -1
    IF mybutton == 1
        do_something
    ENDIF
DLOOP
```

4.5. Variables y asignación

El lenguaje permite definir variables numéricas mediante dos formatos equivalentes.

4.5.1. Asignación directa

```
myVar=5
```

4.5.2. Asignación mediante operador flecha

```
3 -> counter
```

Ambas expresiones son interpretadas internamente como operaciones válidas de asignación. El analizador léxico identifica estos operadores como tokens de tipo EQUAL y ARROW, respectivamente.

4.6. Control de flujo

4.6.1. Bucles

El lenguaje permite ejecutar bloques de instrucciones de forma repetitiva mediante la estructura LOOP.

La sintaxis básica es la siguiente:

```
LOOP <repeticiones>  
DLOOP
```

También se admiten bucles condicionales:

```
LOOP <variable> <condicion> <valor>  
DLOOP
```

Ejemplo de uso:

```
LOOP 3  
    WRITE=myled on  
    WAIT 1  
    WRITE=myled off  
    WAIT 1  
DLOOP
```

Además, es posible crear bucles infinitos utilizando el valor -1 como número de iteraciones.

4.6.2. Condicionales

La estructura IF permite evaluar condiciones lógicas y ejecutar bloques de instrucciones en función del resultado obtenido.

La sintaxis general es la siguiente:

```
IF <variable> <condicion> <valor>  
    <instrucciones>  
ENDIF
```

Ejemplo:

```
myVar=5
```

```
IF myVar > 5  
    WRITE=myled off  
ENDIF
```

```
IF myVar < 3  
    WRITE=myled on  
ENDIF
```

4.7. Control temporal y salida de datos

El lenguaje incorpora instrucciones destinadas tanto al control temporal de la ejecución como a la visualización y almacenamiento de información.

4.7.1. Control temporal

La instrucción WAIT permite pausar temporalmente la ejecución del programa:

```
WAIT <segundos>
```

Ejemplo:

```
WAIT 1
```

Esta instrucción detiene la ejecución durante el número de segundos especificado.

4.7.2. Salida de datos

El lenguaje incluye una instrucción para definir el archivo donde se almacenarán los registros o logs generados por el sistema:

```
FILE=myLogFile
```

Asimismo, la instrucción PRINT permite mostrar información por consola o en la interfaz del sistema, combinando texto y variables:

```
PRINT "Se ha realizado un cambio en el LED"
```

Los registros generados incluyen automáticamente una marca temporal (*timestamp*), permitiendo identificar el momento exacto en el que ocurrió cada evento:

```
[12/08/2026 - 14:01]: "Se ha realizado un cambio en el LED"
```

4.8. Comunicación I2C

El lenguaje incorpora soporte básico para comunicación con dispositivos externos mediante el protocolo I2C.

```
I2C READ 0x76 0xFA 3 -> temp
```

Esta instrucción permite leer datos desde un dispositivo conectado al bus I2C y almacenar el resultado en una variable interna del sistema.

La arquitectura modular del intérprete permite incorporar nuevos periféricos externos sin necesidad de realizar configuraciones complejas ni modificar la estructura principal del sistema.

4.9. Ejemplo completo de programa

A continuación, se muestra un ejemplo que integra las principales funcionalidades del lenguaje:

```
DEVICE=led NAME=myled pin=17
DEVICE=button NAME=mybutton pin=33

LOOP -1
  IF mybutton == 1
    WRITE=myled on
    PRINT "Cambio realizado en el LED on"
  ELSE
```

```

WRITE=myled off
PRINT "Cambio realizado en el LED off"
ENDIF
DLOOP

```

Este ejemplo combina la declaración de dispositivos, estructuras de control, operaciones de escritura y salida de datos.

4.10. Palabras clave y elementos del lenguaje

Con el objetivo de diferenciar claramente las funcionalidades del lenguaje, las instrucciones se clasifican según su propósito dentro del sistema.

Categoría	Elementos
Inicialización de dispositivos	DEVICE
Tipos de dispositivos	LED, BUTTON, BUZZER
Control de dispositivos	WRITE
Estructuras de control	IF, ELSE, ENDIF, LOOP, DLOOP
Control temporal	WAIT
Comunicación	I2C, READ, INIT, SDA, SCL
Salida de datos y registros	PRINT, FILE
Operadores	=, ->, >, <, ==

Tabla 4.1: Clasificación de palabras clave y elementos del lenguaje.

4.11. Consideraciones de diseño

El diseño del lenguaje se ha orientado a la simplicidad, la eficiencia y la cercanía con el hardware, permitiendo su utilización en dispositivos embebidos con recursos limitados.

Las principales decisiones de diseño adoptadas han sido las siguientes:

- **Simplicidad sintáctica:** utilización de instrucciones cortas, directas y fáciles de interpretar.
- **Interpretación en tiempo real:** eliminación de la necesidad de compilación previa para modificar el comportamiento del sistema.
- **Flexibilidad en la asignación de variables:** soporte para distintos estilos de asignación.
- **Modularidad y extensibilidad:** arquitectura preparada para incorporar nuevos dispositivos, operadores y estructuras de control sin modificaciones importantes en el núcleo del intérprete.
- **Bajo coste computacional:** diseño orientado a minimizar el consumo de recursos dentro del dispositivo embebido.

4.12. Futuras mejoras

Aunque el lenguaje desarrollado cubre las necesidades principales del sistema actual, la arquitectura modular del intérprete permite incorporar nuevas funcionalidades de forma sencilla y escalable.

Entre las mejoras contempladas para futuras versiones destacan las siguientes:

- **Acceso directo a sensores internos:** actualmente, sensores integrados en el dispositivo, como la IMU (*Inertial Measurement Unit*), únicamente pueden consultarse mediante aplicaciones internas del firmware. Como futura ampliación, se plantea incorporar instrucciones específicas dentro del DSL que permitan acceder directamente a estos sensores desde los programas interpretados.
- **Incorporación de operaciones aritméticas:** en la versión actual el lenguaje se centra principalmente en operaciones de control y gestión de dispositivos. Futuras versiones incluirán soporte para operaciones matemáticas y expresiones aritméticas, permitiendo trabajar con operadores como suma (+), resta (-), multiplicación (*) y división (/).
- **Mejoras en el sistema de eventos:** se contempla la incorporación de mecanismos orientados a eventos, permitiendo reaccionar automáticamente ante cambios de estado sin necesidad de emplear bucles de supervisión continua.

Capítulo 5

Diseño de la API

5.1. Introducción

El sistema desarrollado incorpora una interfaz HTTP integrada directamente en el dispositivo, permitiendo la interacción con el usuario mediante una aplicación web servida desde el propio ESP32-S3. A diferencia de arquitecturas tradicionales basadas en APIs REST orientadas al intercambio de datos en formato JSON, el enfoque adoptado en este proyecto se centra en la generación y entrega directa de interfaces HTML dinámicas.

De este modo, el microcontrolador no únicamente actúa como servidor de comunicación, sino también como servidor web encargado de construir y servir las distintas páginas que conforman la interfaz de usuario del sistema.

Este enfoque permite reducir la dependencia de servicios externos y simplifica el despliegue del dispositivo, ya que toda la lógica principal de interacción se encuentra integrada localmente en el sistema embebido.

La interfaz web desarrollada proporciona herramientas para:

- Gestionar los archivos almacenados en la tarjeta microSD.
- Editar programas escritos en el lenguaje específico de dominio (DSL).
- Consultar documentación integrada del sistema.
- Supervisar el estado general del dispositivo.
- Interactuar con funcionalidades internas del sistema.

5.2. Configuración Wi-Fi y modos de funcionamiento

El sistema incorpora un mecanismo de configuración Wi-Fi orientado a simplificar la conexión inicial del dispositivo y, al mismo tiempo, limitar accesos no autorizados a la interfaz de administración.

Para ello, el dispositivo dispone de un endpoint específico accesible mediante la ruta `/wifi`. Esta ruta genera dinámicamente una interfaz HTML desde el propio ESP32-S3, permitiendo al usuario introducir las credenciales de la red inalámbrica a la que desea conectar el dispositivo.

Durante esta fase inicial, el dispositivo funciona mediante un punto de acceso propio (Access Point o AP), permitiendo que el usuario se conecte directamente al sistema para realizar la configuración de red desde un navegador web convencional.

Una vez completado el proceso de configuración, el sistema almacena los datos necesarios y cambia automáticamente al modo de producción. En este modo, el endpoint `/wifi` deja de estar disponible, impidiendo que otros usuarios puedan modificar remotamente la configuración de red del dispositivo.

Este enfoque permite añadir una capa adicional de protección, ya que la configuración de acceso inalámbrico únicamente se encuentra habilitada durante el modo de desarrollo o configuración inicial.

El sistema contempla dos modos principales de funcionamiento:

- **Modo desarrollo:** estado inicial del dispositivo. En este modo se habilitan las herramientas de configuración y depuración, incluyendo el endpoint `/wifi`.
- **Modo producción:** estado operativo normal del dispositivo. En este modo los endpoints relacionados con configuración interna quedan deshabilitados, reduciendo la superficie de acceso disponible.

En caso de que el usuario necesite modificar posteriormente la red Wi-Fi configurada, el sistema permite volver temporalmente al modo desarrollo mediante una combinación física de botones. Para ello, uno de los botones integrados debe mantenerse pulsado durante varios segundos, provocando el cambio de modo y habilitando nuevamente las herramientas de configuración.

Este mecanismo evita que la reconfiguración inalámbrica pueda realizarse remotamente de manera continua, obligando a disponer de acceso físico al dispositivo para habilitar nuevamente las funciones de configuración.

No obstante, debe destacarse que este enfoque no protege frente a accesos físicos directos sobre el hardware. Un usuario con acceso físico al dispositivo podría forzar manualmente el cambio de modo y reactivar las funcionalidades de configuración. Debido a las limitaciones propias del prototipo y del alcance del Trabajo de Fin de Grado, no se han implementado mecanismos avanzados de protección física o autenticación adicional.

Funcionamiento del modo Access Point (AP)

El sistema incorpora un mecanismo de funcionamiento basado en modo *Access Point* (AP) como se mencionó anteriormente, orientado a facilitar tanto la configuración inicial del dispositivo como su utilización en entornos sin infraestructura de red disponible.

Este modo se habilita automáticamente en dos situaciones principales:

- Cuando el dispositivo no dispone de una red Wi-Fi previamente configurada.
- Cuando el sistema pierde la conexión con la red inalámbrica almacenada y no consigue restablecerla automáticamente.

En cualquiera de estos casos, el ESP32-S3 genera un punto de acceso inalámbrico propio al que el usuario puede conectarse directamente desde cualquier dispositivo compatible, incluyendo teléfonos móviles, ordenadores portátiles o tablets.

Además, el sistema muestra en pantalla el nombre de la red generada, facilitando al usuario la identificación del punto de acceso al que debe conectarse para acceder localmente a la interfaz web del dispositivo y a los distintos endpoints disponibles.

Este enfoque permite que el sistema pueda utilizarse incluso en escenarios donde no exista una red Wi-Fi externa disponible, manteniendo el acceso a funcionalidades como:

- Consulta de archivos almacenados.
- Edición de programas del DSL.
- Acceso al dashboard principal.
- Visualización de documentación integrada.
- Monitorización básica del dispositivo.

Gracias a esta arquitectura híbrida, el dispositivo puede funcionar tanto integrado dentro de una red local convencional como de forma completamente autónoma, mejorando su portabilidad y adaptabilidad a distintos entornos de uso.

5.3. Arquitectura de la Interfaz Web

La arquitectura implementada se basa en un servidor HTTP ligero ejecutado directamente sobre el ESP32-S3. Cada ruta definida dentro del sistema genera dinámicamente una página HTML específica según la funcionalidad solicitada por el usuario.

En lugar de emplear una separación tradicional entre backend y frontend, el dispositivo integra ambas responsabilidades dentro del propio firmware. Esto permite que el sistema procese internamente los datos y genere directamente las vistas necesarias para el usuario final.

Gracias a este enfoque, se consigue:

- Reducir la complejidad general del sistema.
- Eliminar dependencias externas.
- Facilitar el acceso desde cualquier navegador web.
- Mejorar la portabilidad del dispositivo.
- Mantener una arquitectura autocontenida.

5.4. Rutas principales del sistema

El sistema dispone actualmente de un conjunto reducido de rutas principales orientadas a la gestión del dispositivo y la interacción con el entorno de desarrollo integrado.

Ruta	Función	Descripción
/	Dashboard principal	Página principal del sistema. Centraliza la interacción con el dispositivo y actúa como punto de acceso a las distintas funcionalidades disponibles.
/wifi	Configuración wifi	Genera una interfaz HTML para la configuración de la conexión wifi.
/browser	Explorador de archivos	Genera una interfaz HTML para visualizar y gestionar los archivos almacenados en la tarjeta microSD.
/editor	Editor de texto	Proporciona un entorno web básico para crear y modificar archivos utilizados por el sistema.
/librarie	Documentación DSL	Muestra documentación integrada relacionada con el lenguaje específico de dominio desarrollado para el dispositivo.
/ping	Control de ping interno	Permite activar o desactivar un mecanismo interno de comprobación y monitorización del sistema.

Tabla 5.1: Rutas principales disponibles en el sistema.

5.5. Dashboard principal

La ruta / constituye el núcleo principal de la interfaz web del dispositivo. Desde esta vista se centraliza el acceso al resto de funcionalidades disponibles dentro del sistema.

El dashboard actúa como punto de entrada principal para el usuario, proporcionando acceso al explorador de archivos, editor integrado y documentación del DSL. Además, permite supervisar el estado general del dispositivo y facilitar futuras ampliaciones de la interfaz.

El diseño se ha planteado siguiendo criterios de simplicidad y bajo consumo de recursos, adaptándose a las limitaciones propias de un sistema embebido ejecutado sobre un microcontrolador.

5.6. Gestión de archivos y edición de programas

Uno de los objetivos principales del sistema consiste en permitir la creación y modificación de programas directamente desde el dispositivo, sin necesidad de herramientas externas.

Para ello, se incorporan dos vistas principales:

- **/browser**: interfaz orientada a la exploración y gestión de archivos almacenados en la tarjeta microSD.
- **/editor**: entorno de edición textual integrado que permite modificar programas y configuraciones del sistema directamente desde el navegador.

Ambas funcionalidades trabajan conjuntamente para proporcionar un entorno ligero de desarrollo integrado accesible desde cualquier dispositivo conectado a la red local.

5.7. Documentación integrada del DSL

Con el objetivo de facilitar el uso del lenguaje específico de dominio desarrollado en este proyecto, el sistema incorpora una sección dedicada a documentación accesible desde la ruta `/librarie`.

Esta interfaz proporciona información básica sobre la sintaxis del lenguaje, instrucciones disponibles y ejemplos de uso, permitiendo al usuario consultar rápidamente el funcionamiento del DSL sin abandonar el entorno del dispositivo.

La inclusión de esta funcionalidad refuerza el objetivo principal del proyecto: reducir la complejidad de interacción con sistemas IoT y facilitar el desarrollo de nuevas aplicaciones.

5.8. Sistema de monitorización interna

El sistema incorpora una funcionalidad básica de monitorización accesible mediante la ruta `/ping`. Esta característica permite activar o desactivar un mecanismo interno utilizado para comprobar el estado operativo del dispositivo.

Aunque su implementación es sencilla, este mecanismo resulta útil durante tareas de depuración, desarrollo y validación del sistema.

5.9. Consideraciones de diseño

El diseño de la interfaz web y del servidor HTTP se ha realizado teniendo en cuenta las limitaciones propias de los sistemas embebidos y los objetivos generales del proyecto.

Las principales decisiones de diseño adoptadas han sido las siguientes:

- **Arquitectura autocontenida**: tanto el servidor como la interfaz web se ejecutan directamente sobre el ESP32-S3.
- **Bajo consumo de recursos**: generación de páginas HTML ligeras compatibles con las capacidades del microcontrolador.
- **Simplicidad de acceso**: utilización de interfaces accesibles desde cualquier navegador sin necesidad de software adicional.

- **Modularidad:** posibilidad de incorporar nuevas rutas y funcionalidades sin modificar significativamente la arquitectura principal.
- **Facilidad de uso:** integración de herramientas básicas de edición y gestión dentro del propio dispositivo.

Este enfoque permite disponer de una plataforma ligera y flexible orientada al desarrollo rápido de aplicaciones IoT sobre hardware embebido.

Capítulo 6

Desarrollo del software

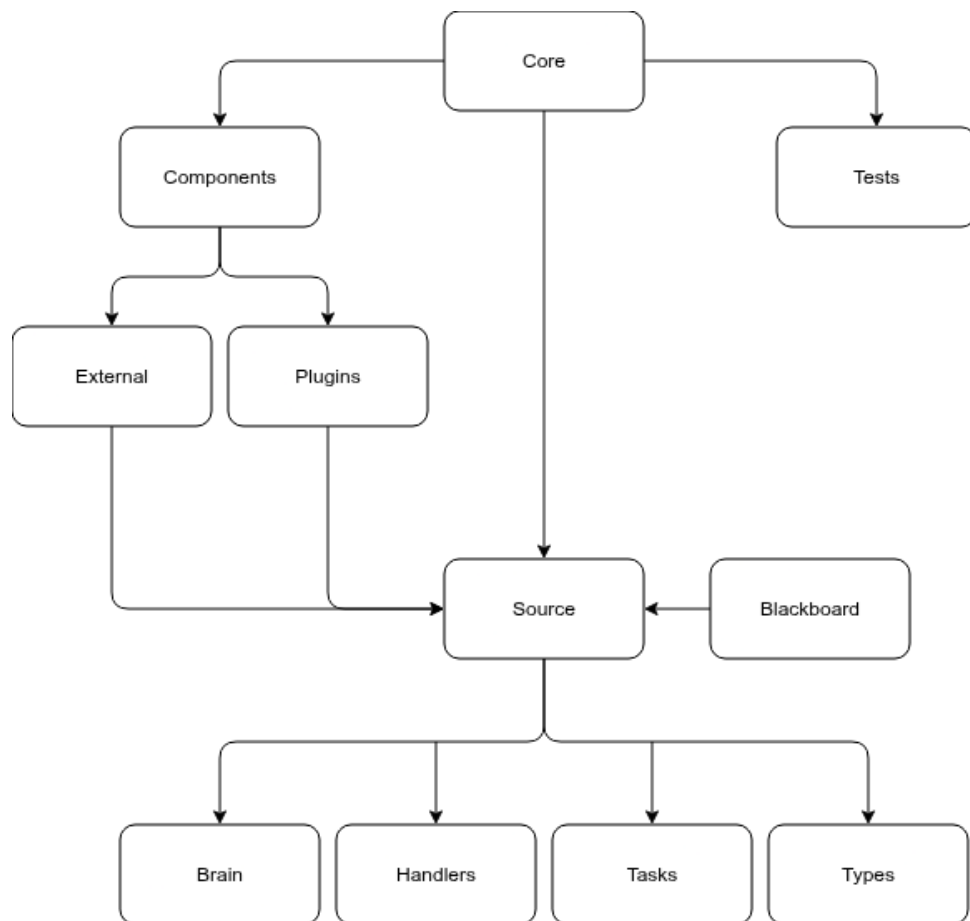


Figura 6.1: Representación conceptual del diseño del software.

El diseño del software se encuentra dividido en diferentes capas y módulos con el objetivo de mantener una arquitectura desacoplada, modular y fácilmente escalable. Todo el código se estructura sobre un núcleo principal denominado *Core*, a partir del cual se ramifican los distintos módulos según su funcionalidad. Este enfoque permite evitar la compilación de código innecesario dentro del dispositivo, reduciendo así el consumo de recursos y facilitando el mantenimiento del sistema.

En el primer nivel de la arquitectura se encuentran los componentes, cuya lógica está desacoplada del sistema principal para permitir su desarrollo y mantenimiento independiente. Estos componentes se dividen en dos grandes grupos: componentes externos y plugins.

Por un lado, los componentes externos corresponden a librerías o código fuente ajeno al desarrollo realizado en este TFG. Actualmente, únicamente se emplea un módulo externo encargado de la gestión de la pantalla o display.

Por otro lado, los plugins agrupan los módulos desarrollados específicamente para este proyecto. Entre ellos se encuentran:

- **Lexer:** encargado del análisis léxico del lenguaje desarrollado.
- **Módulo IO:** proporciona soporte para los distintos dispositivos y periféricos compatibles con el lenguaje.
- **Logger:** sistema centralizado de registros que permite habilitar o deshabilitar mensajes de depuración según las necesidades del dispositivo, reduciendo así la carga de procesamiento.
- **Observable:** módulo que implementa variables observables y sistemas de suscripción. Gracias a este enfoque es posible reaccionar ante cambios de estado sin necesidad de realizar comprobaciones constantes mediante bucles, disminuyendo el consumo computacional del sistema.

En este mismo nivel también se encuentran los tests, completamente desacoplados de la lógica principal. Su función es validar el correcto funcionamiento del lexer y asegurar la estabilidad del sistema durante el desarrollo. Además, están integrados en un flujo automatizado mediante GitHub Actions, que ejecuta los tests en cada cambio enviado al repositorio para garantizar que el sistema siga funcionando correctamente y evitar la introducción de errores en nuevas versiones.

En una capa inferior se sitúa el módulo *Source*, donde reside la lógica principal del sistema. Este módulo hace uso de todos los componentes previamente mencionados y contiene además un sistema *blackboard*, encargado de almacenar las variables principales del dispositivo. Este enfoque permite acceder y modificar el estado global del sistema de forma centralizada, evitando dependencias innecesarias entre módulos y reduciendo la necesidad de incluir cabeceras de otras funcionalidades.

Dentro de *Source* se distinguen cuatro módulos principales:

- **Brain:** contiene la lógica principal del dispositivo. En este módulo se encuentran:
 - La gestión del servidor, que hace uso del módulo *handlers*.
 - La gestión de la conectividad WiFi.
 - El intérprete del lenguaje.
 - El controlador del display, encargado de inicializar y preparar la pantalla mediante el módulo externo correspondiente.
 - El módulo *expander*, responsable de preparar y gestionar la ranura de expansión del dispositivo (I2C).
- **Handlers:** módulo encargado de la API del sistema. Aquí se definen y gestionan todos los endpoints disponibles.
- **Task:** módulo responsable de la creación y gestión de tareas del sistema. Estas tareas utilizan las funcionalidades implementadas en el módulo *Brain*. Entre ellas se encuentran, por ejemplo, la inicialización del servidor o la sincronización entre la lectura de la tarjeta SD y la actualización del display.

- **Types:** conjunto de estructuras de datos y tipos personalizados diseñados para simplificar la lectura y el desarrollo del código, permitiendo representar de manera clara tanto variables como estados internos del dispositivo.

Cabe destacar que el módulo encargado del display está gestionado mediante una máquina de estados simple. Esto permite modificar dinámicamente el comportamiento de los botones principales del dispositivo y controlar en cada momento la información mostrada en pantalla.

Capítulo 7

Diseño Final del Dispositivo

En este capítulo se presenta el diseño conceptual del dispositivo desarrollado, describiendo tanto su estructura física como la organización general de sus componentes internos. El objetivo principal de esta fase ha sido definir una arquitectura modular, flexible y reutilizable, alineada con los requisitos planteados en capítulos anteriores.

Debe destacarse que el diseño presentado tiene un carácter conceptual y orientado a prototipo. Debido a las limitaciones temporales y materiales propias del desarrollo del Trabajo de Fin de Grado, algunos elementos descritos no han podido implementarse físicamente en su totalidad, aunque sí han sido considerados durante la fase de diseño y planificación del sistema.

7.1. Contexto del Diseño

El dispositivo se encuentra basado en una arquitectura centralizada alrededor de un microcontrolador ESP32-S3, encargado de la gestión de comunicaciones, procesamiento de datos, ejecución del intérprete del lenguaje desarrollado y control de los distintos periféricos integrados.

El diseño general del sistema se ha desarrollado siguiendo varios criterios fundamentales:

- Modularidad y capacidad de expansión mediante buses de comunicación estándar.
- Reducción de complejidad para facilitar el uso del dispositivo.
- Flexibilidad para integrar nuevos sensores y periféricos.
- Eficiencia energética y estabilidad operativa.
- Adaptabilidad a diferentes escenarios de monitorización y automatización.

A nivel conceptual, el sistema busca servir como una plataforma reutilizable sobre la que puedan desarrollarse distintas soluciones IoT sin necesidad de rediseñar completamente el hardware base.

7.2. Diseño Físico del Dispositivo

El diseño físico describe la distribución externa y los elementos accesibles para el usuario. La propuesta contempla un dispositivo compacto orientado a facilitar tanto la interacción directa como la integración de módulos adicionales.

La estructura externa del prototipo contempla los siguientes elementos:

- Botón principal de encendido y apagado.
- Pantalla frontal para visualización de información y estado del sistema.
- LEDs indicadores de funcionamiento y conectividad.
- Ranura de expansión para módulos externos.
- Sistema de almacenamiento extraíble mediante tarjeta microSD.
- Salida de audio mediante zumbador integrado.
- Aberturas destinadas a ventilación y disipación térmica.

El diseño planteado prioriza la accesibilidad de los componentes principales y la facilidad de uso, manteniendo un formato reducido y portable.

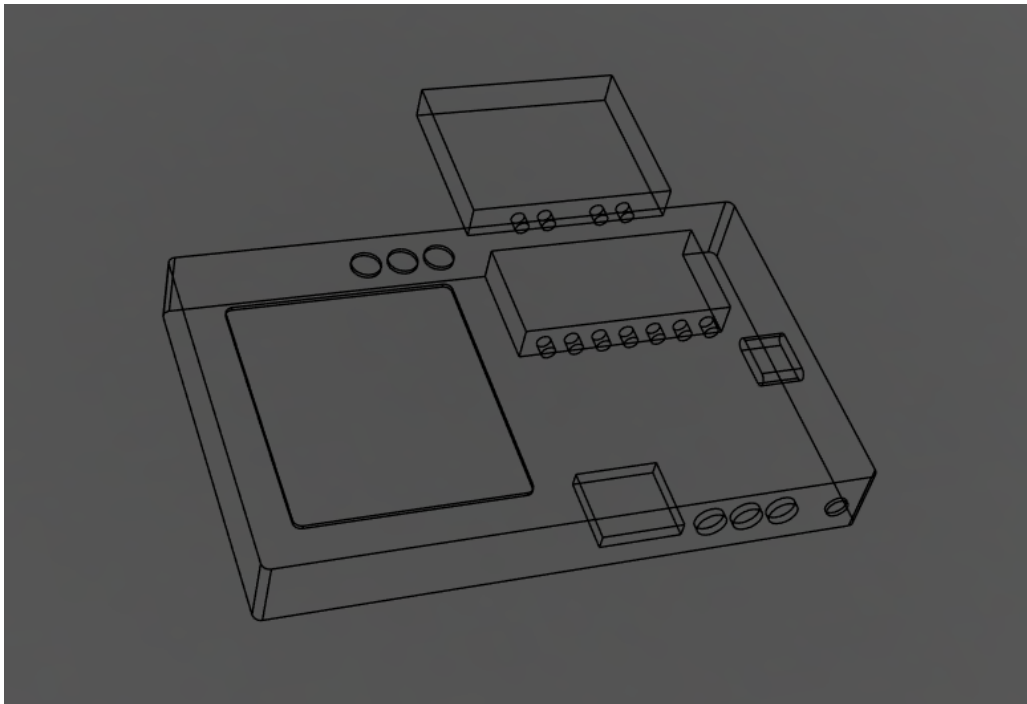


Figura 7.1: Diseño interno del dispositivo. Distribución de módulos, sensores, tarjeta SD, ventilador y núcleo central ESP32-S3.

7.3. Diseño e Integración de Componentes Internos

Este apartado describe la organización conceptual de los principales módulos internos del dispositivo, así como los criterios generales utilizados para su integración.

7.3.1. Estructura interna y módulos principales

La distribución interna se ha planteado con el objetivo de optimizar la organización de conexiones, la capacidad de expansión y la accesibilidad de los distintos módulos del sistema.

La arquitectura interna del dispositivo se compone principalmente de los siguientes bloques funcionales:

- **Núcleo de procesamiento:** microcontrolador principal encargado de la ejecución del sistema, conectividad inalámbrica y control de periféricos.
- **Sistema de sensorización:** conjunto de sensores internos orientados a la captura de información ambiental, movimiento, orientación y variables físicas del entorno.
- **Sistema de almacenamiento:** almacenamiento externo basado en tarjeta microSD para la gestión de aplicaciones, configuraciones y registros del sistema.
- **Sistema de interacción:** pantalla, LEDs indicadores y zumbador destinados a la interacción directa con el usuario.
- **Sistema de expansión:** buses de comunicación preparados para la conexión de módulos externos y periféricos adicionales.
- **Sistema de alimentación:** conjunto de circuitos destinados a la regulación y distribución energética del dispositivo.

La arquitectura modular planteada permite incorporar nuevos componentes sin modificar la estructura principal del sistema, favoreciendo así la escalabilidad y reutilización del dispositivo.

7.3.2. Gestión térmica

Durante la fase de diseño se contempló la incorporación de un sistema de ventilación activa orientado a mejorar la estabilidad térmica del dispositivo en escenarios de alta carga de procesamiento o uso continuado.

La propuesta considera la utilización de mecanismos de refrigeración controlados mediante firmware, capaces de activarse en función de parámetros como la temperatura interna o el estado energético del sistema.

No obstante, este apartado debe entenderse como una propuesta teórica de diseño. Debido a que el prototipo desarrollado no dispone de una carcasa final completamente implementada, no ha sido posible realizar pruebas reales relacionadas con flujo de aire, disipación térmica o comportamiento interno de temperatura.

Por este motivo, las decisiones relacionadas con ventilación y gestión térmica se presentan como una línea conceptual orientada a futuras iteraciones del dispositivo, más que como una funcionalidad validada experimentalmente.

7.4. Comentarios sobre el Diseño

El diseño presentado persigue ofrecer una plataforma IoT flexible, reutilizable y adaptable a distintos entornos de uso. La combinación de modularidad hardware, capacidad de expansión y soporte software mediante un lenguaje específico de dominio permite reducir significativamente la complejidad de desarrollo de nuevas aplicaciones.

Cabe destacar que gran parte de las decisiones descritas corresponden a un diseño conceptual planteado para una versión futura más completa del sistema. Algunas características no han podido implementarse físicamente debido a limitaciones de tiempo, disponibilidad de materiales y alcance propio del Trabajo de Fin de Grado.

Aun así, el trabajo realizado establece una base sólida para futuras ampliaciones tanto a nivel hardware como software, permitiendo evolucionar el dispositivo hacia una plataforma IoT más robusta y completa.

Capítulo 8

Summary and Conclusions

This chapter is compulsory. The memory should include an extended summary and conclusions in English.

Capítulo 9

Presupuesto

9.1. Sección Uno

Tabla 9.1: Presupuesto de Equipos y Licencias

Descripción	Cantidad	Coste (€)
Placa ESP32, WIFI + BLUETOOTH	1	17,80
Expansor de pines digitales	1	4,25
Pantalla color SPI TFT de 2 pulgadas, 176X220	1	19,95
Giroscopio y Acelerometro	1	6,95
Sensor Temperatura y Humedad	1	4,95
Sensor Llama y Gas	1	3,20
Kit de componentes	1	6,90
Subtotal de componente		64

Tabla 9.2: Coste de Mano de Obra

Descripción	Horas	Coste (€)
Precio por Hora		20,00
Total de Horas de Trabajo	300	
Costo Total del Trabajo Humano		6000,00

Tabla 9.3: Coste Total del Proyecto

Descripción	Coste Total (€)
Subtotal de componentes	64,00
Costo Total del Trabajo	6000,00
Coste Total del Proyecto	6064,00

Apéndice A

Título del Apéndice 1

A.1. Algoritmo XXX

Ejemplo de código con coloreado de sintaxis:

```
1 #include <iostream>
2
3 int main()
4 {
5     // Imprime "Hello, world!" en la consola
6     std::cout << "Hello, world!\n";
7     return 0;
8 }
```

A.2. Archivo XXY

Ejemplo de JSON usando el mismo entorno de coloreado de sintaxis:

```
1 {
2     "nombre": "John Doe",
3     "edad": 30,
4     "ciudad": "Nueva York",
5     "hobbies": [
6         "lectura",
7         "jardinería",
8         "ciclismo"
9     ],
10    "empleo": {
11        "título": "Ingeniero de Software",
12        "empresa": "TechCorp"
13    }
14 }
```

A.3. Algoritmo YYY

Este es el clásico entorno verbatim, sin coloreado pero con fuente monoespaciada:

```
/******  
*  
* Fichero .h  
*  
*****  
*  
* AUTORES  
*  
* FECHA  
*  
* DESCRIPCION  
*  
*  
*****/
```

A.4. Algoritmo ZZZ

Ejemplo de entorno para describir algoritmos en pseudocódigo:

Algoritmo A.1: Cálculo del factorial de un número

```
1: Entrada: Un entero  $n$   
2: Salida: El factorial de  $n$   
3: function FACTORIAL( $n$ ) ▷ El factorial de  $n$   
4:   if  $n \leq 1$  then  
5:     return 1 ▷ El factorial de 0 o 1 es 1  
6:   else  
7:     return  $n \times$  FACTORIAL( $n-1$ )
```

Otra forma de describir algoritmos es utilizar entornos `lstlisting` y emplear una sintaxis de pseudocódigo similar a alguno de los lenguajes soportados por este paquete, como Python o Pascal.

Apéndice B

Título del Apéndice 2

B.1. Otro apéndice: Sección 1

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

B.2. Otro apéndice: Sección 2

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.